

Programming with R – CHEATSHEET

Sean McGowan



Getting started

R-ENVIRONMENT

Interface

- Console (bottom-left): where code is executed
- Source (top-left): editing scripts
- Environment (top-right): variables and history
- Files/Plots/Help (bottom-right): figures, documentation

Useful commands

- Check version: `R.version`
- Working directory: `getwd()`, `setwd()`
- Clear workspace: `rm(list = ls())`
- View objects: `ls()`

FILES

- Run a script: click Run
- Run specific lines: highlight and press `<ctrl>/<cmd> + <enter>`
- Comment code: `# a comment`

Comments explain *why* not *what*

Objects

TYPES

- Numeric, logical, character
- Check type: `class(x)`, `str(x)`
- Length/dim: `length()`, `dim()`

Assign variables: `x <- 1`, `TRUE -> y`,
`z = "a"`

COMMON OBJECTS

- Strings: `"Hello world!"`
- Vectors: `c(1,2,3)`, `1:10`
- Matrices: `matrix(1:6, nrow = 2)`
- Lists: `list(a=1, b="x")`
- Data frames: `data.frame(x=1:3, y=4:6)`

OPERATORS

- Arithmetic: `+` `-` `*` `/` `^`
(element-wise execution on vectors)
- Matrix: `%*%` (inner), `%o%` (outer)
- Logical: `<` `>` `<=` `>=`, `&&` (and), `||` (or),
`==` (equals), `!=` (not equals)

INDEXING

- Starts at 1
- Element: `x[i]`
- Range: `x[a:b]`
- Logical indexing: `x[x > 0]`
- Data frame column: `df$col`

Control flow

STRUCTURES

If / else: branched decision making

```
if (x < 0) {  
  ...  
} else if (x == 0) {  
  ...  
} else {  
  ...  
}
```

For: iterate over sequences

```
for (i in 1:5) {  
  ...  
}
```

While: loop until a condition is false

```
while (x < 1) {  
  ...  
}
```

Always *preallocate* objects before loops

Nested loops: loops within loops

Loop control

- `break:` stop loop
- `next:` skip iteration

Functions

DEFINING FUNCTIONS

Inline: short, one-line functions

```
f <- function(x, y) {x + 2*y}
```

Function block: longer, many-lined functions

```
f <- function(x, y) {  
  x + 2*y  
}
```

Anonymous functions: no-name functions within functions

```
function(x) x^2
```

Multiple outputs: `return(list())`

USING FUNCTIONS

Calling

- Positional: `f(1,2)`
- Named: `f(x=1, y=2)`

Defaults: arguments with default values

```
g <- function(a, b, n=2) {a^n - b^n}
```

Apply family: function applied to object

- `apply()`: arrays
- `lapply()`: list output
- `sapply()`: simplified output
- `tapply()`: applied to subset

Variable scope: local, global

Reading and writing files: `readLines`,
`writeLines`, `read.csv`, `write.csv`

USEFUL FUNCTIONS

- `round()`, `floor()`, `ceiling()`
- `abs()`, `sign()`
- `sqrt()`, `exp()`, `log()`, `log10()`

- `sin()`, `cos()`, `tan()`, `sinh()`, `cosh()`,
`tanh()`, `asin()`, `acos()`, `atan()`

Packages

USING PACKAGES

Install & load

```
install.packages("ggplot2")  
library(ggplot2)
```

Use without attaching

```
stats::rnorm(100)  
ggplot2::ggplot()
```

POPULAR PACKAGES

- Plotting: `ggplot2`, `ggvis`
- Data: `tidyverse`
- ML: `tidymodels`, `mlr3`

Debugging and help

Common errors

- Error in `x[i]`, subscript out of bounds
- Error in `fun(...)`, unused argument
- Error in `x/0`, Inf or NaN produced
- Object 'x' not found
- Non-conformable arrays

Debugging tools

- `print()`, `cat()`, `str()`
- `traceback()`, `browser()`, `debug()`,
`recover()`

Documentation and other resources

- `?function`: view help pages for function or package
- `??keyword`: searches all help pages for keyword
- `example(function)`: execute runnable examples for function
- Online resources: CRAN, Stack Overflow, R for Data Science by Wickham and Golemund